Make My Marriage

Multi-Tenant Database Design

PostgreSQL • Spring Boot • React • Marriage as Tenant

Core principle

Each marriage is an independent tenant. Tenant-owned records carry marriage_id, and every request must verify that the authenticated user is authorized for that marriage. Global user identities are kept separate from marriage workspaces.

1. Tenancy model

Use a shared PostgreSQL database and shared tables for V1. The marriages table is the tenant root. marriage_members maps users to marriages and grants roles. The schema supports multiple marriages per user, even if the initial UI focuses on one workspace.

Non-negotiable rules

- users are global identities, not tenant-owned.
- marriage_id is required on every wedding-specific table.
- Child rows should use composite foreign keys (marriage_id, parent_id) to prevent cross-tenant relationships.
- Do not trust a marriage_id from the client until membership and role are checked.
- Query tenant data using both the record ID and marriage_id.

Role	Permissions
ADMIN	Bride/groom; manage settings, members, events, guests, expenses, vendors, photos and moderation.
MANAGER	Family organizer; manage planning data. Restrict sensitive membership and destructive actions.
GUEST	RSVP/photo upload through authorized invitation or upload links; no organizer dashboard access.

2. Relationship overview

```
users (global)
  |-- marriage_members -- marriages (TENANT ROOT)
                          |-- marriage_settings
                          |-- events -- event_tasks
                          |-- guests -- invitations -- rsvps
                          |-- guest_households (optional)
                          |-- expenses
                          |-- saved_vendors -- vendors (global catalog option)
                          |-- photo_albums -- photos
                          |-- photo_upload_links
                          |-- notifications
                          |-- audit_logs
```

3. Core tables

Table	Key columns and constraints	Purpose
users	id UUID PK; email UNIQUE; password_hash; display_name; status; created_at; updated_at	Global account identity.
marriages	id UUID PK; name; wedding_date DATE; timezone; created_by FK users; status; created_at; updated_at; deleted_at NULL	Tenant/workspace root.
marriage_members	marriage_id FK; user_id FK; role; status; invited_by; joined_at; PK(marriage_id,user_id)	Membership and authorization.
marriage_settings	marriage_id PK/FK; theme_key; currency_code; locale; photo_moderation_required; settings JSONB; updated_at	Tenant preferences.
events	id UUID PK; marriage_id FK; name; event_type; starts_at/ends_at TIMESTAMPTZ; venue; address; livestream_url; notes; UNIQUE(marriage_id,id)	Wedding functions.
event_tasks	id UUID PK; marriage_id; event_id; title; description; due_at; status; assigned_user_id; sort_order; FK(marriage_id,event_id)	Checklist items.
guests	id UUID PK; marriage_id; household_id; full_name; email/phone; side; relationship_label; invite_status; UNIQUE(marriage_id,id)	Guest list.
guest_households	id UUID PK; marriage_id; household_name; contact fields; UNIQUE(marriage_id,id)	Optional family grouping.
invitations	id UUID PK; marriage_id; guest_id; event_id NULL; token_hash; channel; sent_at; expires_at; status; composite FKs	Invitation and secure RSVP link.
rsvps	id UUID PK; marriage_id; invitation_id; guest_id; event_id; response; party_size; responded_at; UNIQUE(marriage_id,guest_id,event_id)	Response per guest and event.
expenses	id UUID PK; marriage_id; event_id NULL; category; description; vendor_id NULL; amount NUMERIC(12,2); amount_paid NUMERIC(12,2); currency_code; due_date; notes; created_by	Budget and payment tracking.
vendors	id UUID PK; name; category; city; contact fields; website_url	Global vendor catalog, if shared.
saved_vendors	id UUID PK; marriage_id; vendor_id; status; quoted_amount; contact_notes; UNIQUE(marriage_id,vendor_id)	Marriage-specific vendor shortlist.
photo_albums	id UUID PK; marriage_id; event_id NULL; name; visibility; UNIQUE(marriage_id,id)	Photo groups.
photo_upload_links	id UUID PK; marriage_id; album_id; token_hash; expires_at; max_uploads; uploads_used; revoked_at	QR-friendly upload link.

Table	Key columns and constraints	Purpose
photos	id UUID PK; marriage_id; album_id; object_key; mime_type; size_bytes; moderation_status; moderated_by; created_at; deleted_at	DB stores metadata; file bytes live in private object storage.
notifications	id UUID PK; marriage_id; recipient; type; channel; status; payload JSONB; sent_at	Notification/outbox tracking.
audit_logs	id UUID PK; marriage_id; actor_user_id; action; entity_type; entity_id; metadata JSONB; created_at	Record important actions. Avoid secrets in metadata.

4. Tenant-safe foreign keys and validation

For tenant-owned parent tables, add UNIQUE(marriage_id,id). Child tables should reference the pair (marriage_id, parent_id), not only the parent's ID. This prevents attaching a task from Marriage A to an event in Marriage B.

```
-- Example pattern
CREATE TABLE events (
  id UUID PRIMARY KEY,
  marriage_id UUID NOT NULL REFERENCES marriages(id),
  name TEXT NOT NULL,
  starts_at TIMESTAMPTZ,
  ends_at TIMESTAMPTZ,
  UNIQUE (marriage_id, id),
  CHECK (ends_at IS NULL OR starts_at IS NULL OR ends_at >= starts_at)
);

CREATE TABLE event_tasks (
  id UUID PRIMARY KEY,
  marriage_id UUID NOT NULL,
  event_id UUID NOT NULL,
  title TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'TODO',
  FOREIGN KEY (marriage_id, event_id)
    REFERENCES events (marriage_id, id)
);
```

Indexes to add early

- marriage_members(user_id,status)
- events(marriage_id,starts_at)
- event_tasks(marriage_id,status,due_at)
- guests(marriage_id,full_name)
- expenses(marriage_id,due_date) and expenses(marriage_id,category)
- photos(marriage_id,album_id,moderation_status,created_at)
- invitations(marriage_id,guest_id,status) plus unique index on token_hash
- audit_logs(marriage_id,created_at)

Use UUID identifiers, NUMERIC for money, TIMESTAMPTZ for instants, DATE for calendar dates, and Flyway versioned SQL migrations. Define ON DELETE behavior explicitly.

5. Authorization flow

```
1. Authenticate with Spring Security.
2. Resolve marriageId from the API route.
3. Verify active marriage_members row for (marriageId, currentUserId).
4. Check role/permission for the operation.
5. Query/update records using marriage_id = authorized marriageId.
6. Verify parent and child belong to the same marriage.
7. Reject unauthorized access; avoid leaking other tenants' records.
8. Audit important changes and photo moderation.
```

Implementation guardrails

- Centralize checks in TenantAuthorizationService or equivalent.
- Use repository methods such as findByIdAndMarriageId(id, marriageId).
- Derive tenant context from the authorized route, not from a body field.
- Never accept a role supplied by the browser as proof of permission.
- Guest tokens must be hashed, scoped, expiring, revocable and rate-limited.
- Keep photos private until approved; validate MIME type, size and upload limits.
- PostgreSQL Row-Level Security can be added later as defense in depth, but does not replace application authorization.

6. API route structure

```
POST   /api/v1/auth/register
POST   /api/v1/auth/login
GET    /api/v1/me/marriages
POST   /api/v1/marriages
GET    /api/v1/marriages/{marriageId}
GET/POST       /api/v1/marriages/{marriageId}/events
GET/PATCH/DEL  /api/v1/marriages/{marriageId}/events/{eventId}
GET/POST       /api/v1/marriages/{marriageId}/guests
GET/POST       /api/v1/marriages/{marriageId}/expenses
GET/POST       /api/v1/marriages/{marriageId}/vendors
GET/POST       /api/v1/marriages/{marriageId}/albums
GET/POST       /api/v1/marriages/{marriageId}/photos
POST           /api/v1/public/rsvp/{token}
POST           /api/v1/public/photo-upload/{token}
```

Public tokens should authorize only the specific RSVP or photo-upload action, never general organizer APIs.

7. Java package structure

```
com.makemymarriage
  auth/              User, AuthController, JwtService
  tenancy/           Marriage, MarriageMember, TenantAuthorizationService
  events/            Event, EventTask
  guests/            Guest, Invitation, Rsvp, GuestHousehold
  expenses/          Expense
  vendors/           Vendor, SavedVendor
  photos/            PhotoAlbum, PhotoUploadLink, Photo
  notifications/     Notification, EmailService
  audit/             AuditLog
  common/             ApiExceptionHandler, config, base types
```

8. Implementation phases and security tests

Phase	Deliverable
1	Users, marriages, membership, login, create marriage, tenant checks.
2	Events, tasks, settings, dashboard.
3	Guests, invitations, RSVPs, expenses.
4	Vendor catalog/shortlist, albums, upload links, photo moderation.
5	Audit trail, rate limits, token expiry/revocation, backups and tenant-isolation tests.

Must-pass tests

• A user in Marriage A cannot read, edit or delete Marriage B data.
• A task cannot reference an event in another marriage; RSVP guest/event references must match the same marriage.
• A GUEST cannot call organizer-only APIs, even if they know a UUID.
• Revoked/expired tokens fail; removed membership blocks future calls.
• Expense amounts are validated server-side; photo uploads are private and pending moderation by default.

9. Decisions to confirm

1. Guest accounts: recommended V1 is organizer accounts plus secure token-based RSVP/photo actions; full guest login can come later.
2. Vendor catalog: recommended global vendors + marriage-specific saved_vendors. If vendor data must be private to one wedding, add marriage_id to vendors.
3. Multiple weddings: schema supports it through marriage_members, even if the first UI defaults to one workspace.
4. Retention: decide soft deletion, export, photo retention and audit-log retention before production.

10. References

PostgreSQL constraints: https://www.postgresql.org/docs/current/ddl-constraints.html
PostgreSQL Row-Level Security: https://www.postgresql.org/docs/current/ddl-rowsecurity.html
Spring Security: https://docs.spring.io/spring-security/reference/index.html
Flyway: https://documentation.red-gate.com/flyway